Why This Problem Is Hard

A High-Level Guide to the $\mathcal{ALCI}_{RCC5}$ Decidability Challenge

Michael Wessel — with Claude (Fable 5, Anthropic)

July 2026

A plain-language companion to the ALCI_RCC5 decidability project. No symbols, no proofs — just the ideas, and an honest account of why the central question has stayed open across dozens of rounds. If you have never seen a description logic, start here.

---

1. The question, in one breath

We have a little language for describing **regions in space** — think of blobs, disks, countries, rooms — and how they fit together. You can say things like "every region touching the kitchen also touches the hallway," or "there is a park that is discrete from every building." The question is embarrassingly simple to state and stubbornly hard to answer:

> **Is there an automatic procedure that, given any such description, always halts and correctly says whether the description is satisfiable — whether *some* arrangement of regions fits it?**

"Satisfiable" just means *consistent*: not self-contradictory. "Every region is discrete from itself" is unsatisfiable (a region always equals itself). Most descriptions are subtler, and the subtle ones are where the difficulty lives.

The catch is that a consistent description might only be satisfied by an **infinite** arrangement — infinitely many regions. A computer can't draw infinitely many regions. So the real question becomes: *can we always represent the infinite arrangement by a finite blueprint that a computer can check?* Sometimes yes, sometimes the blueprint itself is forced to be infinite, and telling the two cases apart is the whole game.

---

2. The five ways two regions can relate

Everything is built from **five basic relationships** between two regions. This is called RCC5. Picture two blobs:

- **DR** — *discrete*: they don't share any point (two separate disks).
- **PO** — *partial overlap*: they share some area, but neither is inside the other (two overlapping disks, like a Venn diagram).
- **PP** — *proper part*: one is strictly inside the other (a coin on a table — the coin is a proper part of the table).

- **PPI** — the reverse of PP (the table contains the coin).
- **EQ** — *equal*: they're the same region.

Two facts about these five will matter enormously later:

1. **They're exhaustive and exclusive.** Any two distinct regions stand in *exactly one* of these relationships. There's no "sort of" — pick two blobs and the answer is one of the five.
2. **They compose.** If you know how A relates to B, and how B relates to C, that *constrains* how A relates to C. Example: if the coin is inside the table (PP), and the table is discrete from the moon (DR), then the coin is discrete from the moon (DR) — you didn't have to look at the coin and the moon directly; it was forced. This "composition" is the engine of the whole subject, and — as we'll see — it is forced in some directions but wide open in others. That asymmetry is the crack the whole problem falls into.

---

3. The model: a forest of nested regions with cross-links

The central idea of this project — and the reason the author's original intuition has stayed useful across twenty-plus rounds — is a particular way of *shaping* the arrangement of regions, called the **split-forest model**. It's worth understanding because both the hope and the difficulty come straight from its shape.

Organize the regions along **two axes**:

- **The vertical axis — nesting.** PP and PPI (inside / contains) behave like a **family tree**. A region, its bigger container, that container's bigger container, and so on. Draw the big regions near the top and their parts hanging below, and the "inside-of" relationships become tree branches. This is a clean, well-understood shape.
- **The horizontal axis — side-by-side.** DR and PO (discrete / overlap) are the relationships *between* things that aren't nested in each other — cousins in the family tree, sitting side by side. These are the "cross-links."

So the picture is a **family tree of nested regions (vertical), with cross-links between cousins (horizontal).** Almost everything about why the problem is hard is captured by one sentence: **the vertical axis is tame and the horizontal axis is wild.**

---

4. The tame axis: infinite ladders and the loop trick

Suppose your description forces an endless vertical ladder: this region has a bigger container, which has a bigger container, forever. That's an infinite arrangement — a computer can't draw it.

But there's a classic rescue, and it *works*: the **loop trick** (technically, "blocking"). Since the language can only say finitely many things, eventually two rungs of the ladder look **exactly alike** — same description, same demands. When that happens, you stop drawing new rungs and just *loop back*: "the rest continues like this." A finite diagram with a loop stands in for the infinite ladder.

One subtlety, and it's a real trap the project fell into and climbed out of: when you loop, you must **not** say the two look-alike rungs are the *same region* (EQ). If you glue a rung to an earlier one, you'd be saying a region is a proper part of itself, which is nonsense. The loop is a bookkeeping

device — "continue like this" — not a claim that the ladder actually closes into a ring. Get that wrong and everything collapses; get it right and infinite ladders become finite blueprints.

Bottom line: the vertical axis is solved. Loops tame it. If nesting were the only thing going on, this would be an easy, well-known result.

---

5. The wild axis: why "discrete" only propagates one way

Now the horizontal axis, and the exact asymmetry that makes the problem hard.

Recall composition forces some relationships. Here's the crucial pair:

- **Into the parts (deterministic).** If a big region is discrete from a faraway blob, then *every region inside it* is also discrete from that blob — forced, no choice. (A coin inside a table that's far from the moon is also far from the moon.) "Discrete" rains **down into the parts** automatically.
- **Into the wholes (a free choice).** The reverse does *not* hold. If a small region is discrete from a blob, its bigger containers might be discrete from the blob, might overlap it, might swallow it — all three are possible. Being bigger, a container can reach out and touch things the small region didn't. "Discrete" does **not** propagate up into the wholes; there the arrangement gets to *choose*.

This one-directional determinism is the source of both the hope and the heartbreak. It's genuinely powerful — it's why the split-forest works at all — but it's only *half* a determinism. And every failed proof in this project's history failed by quietly assuming the other half was also forced when it wasn't.

---

6. Shadows: the good news hiding inside the bad

Here's the beautiful consequence of that downward determinism. Suppose a region has to be discrete from some blob. Then all of *its* parts inherit that "discrete" relationship **for free** — you didn't choose it, the composition forced it. We call such a forced, inherited relationship a **shadow**.

Shadows are wonderful for finiteness, because a shadow doesn't need to be *stored* or *checked* independently — it's recoverable from the relationships that caused it. If a blueprint has ten thousand shadow relationships, they cost nothing; they're implied.

So the real measurement of "how big is the blueprint" is not the total number of relationships a region has — it's the number of **live** (non-shadow, genuinely chosen) relationships. Call that the **width**. The whole decidability question comes down to:

> **Does the width stay bounded, or can a description force it to grow without limit?**

If width is always bounded, the blueprint is always finite, and the problem is decidable. If some description forces unbounded width, this whole approach fails for that description.

This is the open problem the project calls **F6**.

---

7. A concrete near-miss (and why it teaches the right lesson)

Recently we tried hard to *break* F6 — to build a description that forces unbounded width. The construction (nicknamed the "alternating-color gap tower") builds a vertical ladder where each rung has a side-blob, cleverly arranged so that one special region at the bottom — call it the **spy** — is forced to be discrete from *every* side-blob up the ladder. We proved, by machine, that the spy really is forced to have more and more discrete neighbors as the ladder grows: no clever rearrangement can avoid it.

For a moment it looked like a counterexample. Then we measured *what kind* of width it was — and every one of those growing relationships turned out to be a **shadow**. The spy's connection to each faraway blob is forced by composition down the ladder; none of it is a live, independent choice. The **live** width stayed flat — bounded — no matter how tall the ladder got.

So the attack failed, but it failed in the most informative way possible: it showed that the natural way to make a region "see" unboundedly many things produces only shadows. The five-relation composition table seems to *conspire* to convert forced structure into shadows. That's not a proof that width is always bounded — but it's real evidence, and it sharpens exactly what a genuine counterexample would need: not a region with many neighbors (always shadows), but a genuinely **live crowd** — many regions packed into one spot whose mutual relationships are all free choices that can't be simplified or merged away.

---

8. Why the obvious argument for "width is bounded" doesn't close

Here is the intuition almost everyone reaches (and it's a good one): *the language has only finitely many things to say, so it can only distinguish finitely many kinds of region. Sooner or later you must reuse an old region instead of inventing a new one — you "run out of colors." So the width can't grow forever.*

This is exactly right on the **vertical** axis — it's the loop trick, and it's why ladders are tame. The trouble is that it does **not** automatically carry over to the **horizontal** axis, and the reason is subtle enough that it has fooled every attempt:

- **The "run out of colors" clock is nesting depth.** Each time you climb the ladder, you use up a little of the description's finite complexity, so eventually it repeats and you loop. The clock that "runs out" is *vertical depth*.
- **But horizontal crowds don't climb the ladder.** A bunch of side-by-side regions all sit at the *same* nesting depth. Adding one more to the crowd doesn't tick the vertical clock at all. So the argument that "you run out of power going up" says nothing about "how wide the crowd at one level can get."
- **And here's the vicious circle.** To *reuse* an old region instead of making a new one, it's not enough that they be described alike. Because every region relates to every other (the arrangement is a complete network), reusing a region means committing to how it relates to *everything already built* — and that new commitment can clash with the composition rules even when the descriptions match. So the thing you'd need to bound isn't just "how many descriptions" (finite, easy) but "how many descriptions **together with their web of relationships to the current crowd**" — and *that* number depends on how big the crowd is, which is the very thing you're trying to bound. The argument chases its own tail.

Breaking a circle like this needs a **measure that shrinks as the crowd grows** — a horizontal clock. Every attempt so far has reached for the *vertical* clock (nesting depth), which provably doesn't move when the crowd widens. That mismatch — vertical clock, horizontal problem — is F6 in one sentence, and it's why a dozen "here's a proof that width is bounded" arguments have all sprung the same leak (technically: reused regions spawn fresh demands at the *same* depth, so the recursion never bottoms out).

Our honest read: width is *probably* bounded — the shadow evidence points that way, and no one has built a genuine live crowd that grows — but "probably" is not "proved," and the missing piece is a genuinely new *horizontal* measure, not another dressing-up of the vertical one.

---

9. The punchline: the counterexample and the proof are the same question — and so is *un*decidability

Recently we chased the missing piece from the other side: instead of trying to *prove* the crowd stays small, we tried hard to *build* a description that forces a crowd to grow forever. Following that all the way to the floor turned up the most clarifying thing in this whole story, and it's worth stating plainly.

First surprise: the scaffolding is free. You might think the hard part is whether a giant side-by-side crowd can *exist* at all. It can, trivially. Take any collection of blobs and, for each pair, just declare them either overlapping or discrete — any pattern you like. Every such pattern is a perfectly legal spatial arrangement (this is guaranteed by the same "patchwork" property the whole project leans on). So a crowd of a thousand distinct side-by-side regions is no problem to *draw*. The obstacle is not "can such a crowd exist."

Second surprise: the *language* can't force one — for the same reason ladders loop. Recall the loop trick that tamed the vertical axis: run a nesting-ladder long enough and, with only finitely many things to say, two rungs eventually look identical, so you loop instead of continuing forever. The exact same pigeonhole applies *sideways*. Try to force an endless side-by-side chain of blobs, and two of them eventually carry the same description with compatible surroundings — at which point nothing stops you from declaring them *the same blob* and closing the chain into a small loop. Side-by-side chains block just like nested ones. So the language cannot force a runaway crowd merely by *asking* for more and more blobs; the crowd folds back on itself.

This last point is no longer just a hunch. An outside reviewer of this work (itself an AI, from a different lab) checked it *exhaustively*: following any chain of "how this relates to that relates to the next," and asking when such a chain can ever loop back on itself, one finds that **every loop is a nesting loop** — there is no purely *sideways* loop that resists folding. Put plainly: a sideways chain always either folds into a small cycle, or it stops being sideways and turns into a nesting chain (which the loop trick already tames). That closes off a whole family of "maybe the crowd can grow this way" attempts in one stroke. It does *not* settle the real question — that is about *width*, not chains — but it is a clean, checkable fact that hardens exactly this leg of the argument.

What it would take. To stop the fold, every blob in the crowd would need a permanent, *unique address* — a fixed coordinate — so that no two could ever be quietly identified. That's rigid coordination: a spatial grid, a ruler, a coordinate system. And here is the wall: **the five relations are too loose to assign addresses.** We checked the entire relationship-composition table exhaustively. The *only* combinations that pin a relationship down to a single forced answer

all run through the *nesting* (vertical) axis — and nesting is exactly the axis that produces free "shadows," not live crowd. On the side-by-side axis nothing is ever forced to a single value; "overlap" in particular is *never* a forced outcome of anything. The language simply has no lever to nail a blob to a fixed sideways coordinate.

So the two questions collapse into one. "Build a description that forces a runaway crowd" and "prove no description can" are not two problems — they are the single question: *can this language force rigid, unbounded, side-by-side coordinates?* If yes, width is unbounded and this whole approach fails; if no, width is bounded and the problem is decidable. One well-posed question, two directions.

And now the real punchline. That *same* question is why nobody has managed to prove the problem **undecidable** either. To prove a logic undecidable, you typically make it encode an unlimited computation — which needs an unlimited, rigidly-coordinated grid to write the computation on. Twenty years ago it was already noticed that this language *cannot* pin such a grid: the relations are too loose to force the "coordinates line up" condition that a grid requires. That is the very same looseness we just ran into. So the decades-long failure to prove the problem *decidable* and the decades-long failure to prove it *undecidable* are **not two separate mysteries — they are one wall seen from two sides.** Both come down to a single fact about these five relationships: **they are too loose to pin rigid side-by-side structure.** Bound that looseness one way and you get decidability; exploit it the other way and you get undecidability; and the reason neither has happened is that the looseness sits exactly on the knife's edge between them.

We did not resolve it — nobody has in twenty years. But the attempt turned a vague "the horizontal axis is hard" into a crisp, two-sided target that a future solver (human or machine) can aim a single argument at. That, again, is the kind of durable clarity this project produces even while the headline question stays open.

---

10. The second open piece: uniformization (W2')

There's a smaller sibling problem worth a sentence. When you compress a real arrangement into a finite blueprint, you label each region with a finite "summary" (its description plus a little local relational information). **Uniformization** is the claim that regions with the same summary are truly interchangeable — that the blueprint captures everything that matters. We measured this and it holds about 98.6% of the time at the coarsest summary; the other 1.4% needs a slightly richer summary. Crucially, getting this wrong can only ever make the procedure *too cautious* (reject something that was actually fine) — it can never make it accept something false. So W2' threatens completeness, never correctness, and it comes with a known repair menu. It's the manageable one.

---

11. The meta-story: what this project actually is

Zoom out. The technical question — decidable or not — is still open. But the *project* has produced something real along the way, and it's worth stating plainly.

- **An adversarial method.** Every proposed proof was handed to a fresh, skeptical reviewer whose job was to break it. Across roughly a dozen rounds, all but one review found a genuine flaw — usually a place where an argument *assumed* the hard thing instead of establishing it. A recurring lesson: a **cold** reviewer (one with no memory of how the proof was built)

consistently found flaws that a warm reviewer, primed by the construction, missed. That is a general lesson about verifying hard arguments, not just about this problem.

- **A machine-checked core.** The most recent phase rebuilt the sound half of the argument inside a **proof assistant** (Lean) — software that mechanically checks every logical step and refuses to accept a gap. The result is a large development with *zero* unproven steps for the parts it covers: the pipeline from "finite blueprint" to "actual arrangement of regions satisfying the description" is now verified by machine, end to end, including the fact that the blueprint's arrangement is a genuine RCC5 world and that it models the logic. What the machine does **not** yet contain is the one hard theorem — F6, bounded width — which is *stated* precisely but deliberately left unproven, because proving it is the open mathematics, and faking it with an assumption would defeat the point.
- **A precise map of the frontier.** Perhaps the most durable output: we now know *exactly* where the difficulty is. It is not spread across the whole problem — it is concentrated into one crisp statement (horizontal width stays bounded) plus its small sibling (uniformization). Everything else is either settled or mechanically checked. A future solver — human or machine — doesn't have to understand the whole edifice; they have to crack one well-posed lemma about horizontal crowds.

That last point is the real deliverable if the decidability question itself never resolves: **the problem has been reduced, honestly and checkably, to its irreducible core, and that core has been explained plainly enough to hand to someone new.** Twenty rounds ago the difficulty was a fog. Now it has an address.

---

12. The one-paragraph summary for the person in a hurry

Regions in space relate in five ways, and those relations *compose* — but composition forces things only in one direction (down into parts, not up into wholes). That asymmetry splits every arrangement into a tame vertical axis (nesting, which infinite-loops away neatly) and a wild horizontal axis (side-by-side crowds). The whole decidability question is whether those horizontal crowds must stay small. They *look* like they must — every attempt to force a big crowd produces only "shadow" relationships that cost nothing — but the clean argument for it keeps failing because it measures the wrong axis: it uses nesting-depth to bound crowd-width, and crowds don't add depth. Chasing it from the other side revealed why it's so stubborn: a big crowd is easy to *draw* but seems impossible to *force*, because side-by-side chains loop back on themselves just like nested ones — and stopping the loop would need rigid coordinates the five relations are too loose to pin. So "build a runaway crowd" and "prove none exists" turn out to be one question, and it's the very same looseness that stops anyone proving the problem *undecidable* (you can't pin a computation-grid either). The project has machine-verified everything *except* that one bounded-crowd fact, stated it precisely, gathered real evidence it's true, and shown it to be the single knife's-edge question sitting between decidability and undecidability. Whether it falls one way or the other is the open problem; that we now know it's the *only* open problem — one fact, two faces — is the achievement.

This document is a companion to the technical write-ups in `papers/` and the machine-checked development in `formal/Round19Transport.lean`. It is deliberately informal; where it and the formal artifacts disagree, the formal artifacts win.